

CLASE: Workshop - Python

¿Por qué Python para el análisis de Datos?

Python es la herramienta líder en análisis de datos debido a su **sintaxis sencilla y legible**, su **amplio ecosistema de bibliotecas gratuitas (Pandas, NumPy, Scikit-learn)**, **alta versatilidad**, y **gran comunidad de soporte**. Es ideal para manipular grandes conjuntos de datos, automatizar tareas, y realizar Machine Learning, superando limitaciones de herramientas como Excel.

Razones clave para elegir Python

Python es la opción estándar para científicos y analistas de datos por su flexibilidad para realizar desde estadísticas básicas hasta modelos complejos de Inteligencia Artificial, permite:

- **Curva de aprendizaje suave:** Su sintaxis es similar al inglés, facilitando su adopción para principiantes.
- **Bibliotecas potentes:** Ofrece herramientas especializadas para cada etapa: Pandas para manipulación de datos, NumPy para computación numérica, Matplotlib/Seaborn para visualización, y Scikit-learn para Machine Learning.
- **Manejo de Big Data:** Capacidad eficiente para procesar grandes volúmenes de datos que superarían la capacidad de hojas de cálculo.
- **Automatización y Repetibilidad:** Permite automatizar flujos de trabajo repetitivos y garantizar que el mismo análisis produzca los mismos resultados.
- **Versatilidad e Integración:** Se integra con SQL, Excel, APIs y otras plataformas de BI, lo que permite un flujo de trabajo completo desde la extracción hasta la visualización.

KIT DE CLASE: Librerías Python para Análisis de Datos

Introducción

El ecosistema Python para análisis de datos es amplio y diverso. Existen múltiples librerías especializadas en diferentes etapas del proceso analítico. Conocer **qué herramienta utilizar en cada etapa** es fundamental para resolver problemas de manera eficiente, optimizando recursos y tiempo.

Este apartado presenta una clasificación funcional de las librerías más utilizadas, con una breve descripción conceptual de su aplicación y criterios orientadores para su elección.

Carga de Datos

Librerías que permiten importar datos desde archivos locales, APIs o páginas web.

- **pandas:** lectura de archivos CSV, Excel, SQL. Ideal para datasets pequeños y medianos.
- **dask:** permite leer archivos en paralelo, útil para archivos grandes (superiores a la RAM disponible).
- **vaex:** optimizado para lectura *out-of-core*, muy eficiente para data en disco.

- **requests:** permite acceder a datos vía APIs REST.
- **beautifulsoup4:** útil para extraer contenido de páginas HTML estáticas.
- **selenium:** automatiza navegación web, ideal para páginas con JavaScript dinámico.

Criterio de elección:

- Tamaño del dataset
- Fuente del dato (archivo, web, API)
- Necesidad de automatización

Limpieza y Transformación de Datos

Herramientas para organizar, filtrar y preparar los datos para su análisis.

- **pandas:** permite operaciones como renombrar columnas, filtrar, agrupar, unir datasets.
- **polars:** alternativa más rápida y eficiente para grandes volúmenes (multihilo).
- **datatable:** buena opción para flujos tabulares en memoria que requieren alta velocidad.

Criterio de elección:

- Tamaño y estructura del dataset
- Necesidad de rendimiento
- Compatibilidad con otras librerías

Análisis Exploratorio de Datos (EDA)

Librerías que permiten explorar rápidamente un dataset para detectar patrones, errores o anomalías.

- **pandas-profiling:** genera un informe automatizado con estadísticas y visualizaciones.
- **sweetviz:** permite comparar datasets (ideal para train/test).
- **autoviz:** facilita el análisis exploratorio con mínima configuración.

Criterio de elección:

- Necesidad de resumen automatizado
- Comparación entre conjuntos
- Facilidad de lectura visual

Visualización de Datos

Herramientas para construir gráficos que comuniquen visualmente los hallazgos del análisis.

- **matplotlib:** base de gráficos 2D. Muy versátil pero más técnico.
- **seaborn:** gráficos estadísticos con mejor estética que matplotlib.
- **plotly:** gráficos interactivos para entornos web, dashboards o notebooks.

Criterio de elección:

- Nivel de interactividad requerido
- Estética frente a control detallado
- Integración con otras herramientas

Modelado Estadístico y Aprendizaje Automático

Librerías para construir modelos predictivos o explicativos a partir de datos.

- **statsmodels**: orientado a modelos estadísticos clásicos (regresión, ARIMA, etc.).
- **scikit-learn**: el estándar para machine learning supervisado y no supervisado.
- **xgboost / lightgbm**: algoritmos de boosting para clasificación y regresión con alto rendimiento.
- **keras / pytorch**: frameworks para redes neuronales y deep learning.

Criterio de elección:

- Tipo de problema (estadístico vs predictivo)
- Complejidad del modelo requerido
- Volumen de datos y rendimiento deseado

Series Temporales

Librerías diseñadas para análisis y pronóstico de datos en el tiempo.

- **statsmodels**: modelos clásicos como ARIMA o SARIMA.
- **prophet**: creado por Meta, simple e intuitivo para forecasting.
- **tsfresh**: extracción automática de features desde series.

Criterio de elección:

- Nivel de experticia del analista
- Tipo de output deseado (modelo o features)
- Facilidad de interpretación

Procesamiento Big Data

Librerías que permiten trabajar con datasets muy grandes (más allá de la RAM disponible).

- **dask**: paraleliza operaciones tipo pandas en múltiples núcleos.
- **pyspark**: interfaz Python para Spark, ideal para clusters y entornos distribuidos.
- **vaex**: permite procesar grandes volúmenes desde disco con alta eficiencia.

Criterio de elección:

- Infraestructura disponible (local vs cloud)
- Tamaño real del dataset
- Nivel de paralelismo requerido

Conclusión

Dominar el uso de estas librerías implica no solo saber escribir código, sino **tomar decisiones informadas** sobre cómo encarar un problema de datos. Un analista competente no sólo responde “cómo”, sino también “con qué y por qué”.

Tabla de librerías con papers / referencias

Librería	Paper / Referencia	Enlace	Utilidad
pandas	Wes McKinney (2010)	pandas Paper	Manipulación y análisis de datos en tablas (DataFrames). Lectura de CSV/Excel/SQL, limpieza y transformación.
numpy	Harris et al. (2020)	Nature Paper	Operaciones matemáticas y manejo eficiente de arreglos multidimensionales (vectores y matrices).
polars	No paper formal (documentación oficial)	Polars Docs	Alternativa a pandas optimizada en Rust. Muy rápida para análisis tabular en datasets grandes.
datatable	No paper formal (documentación oficial)	datatable Docs	Manejo eficiente de grandes datasets en memoria. Similar a pandas pero más rápido en algunos casos.
matplotlib	Hunter (2007)	Matplotlib Paper	Visualización básica: gráficos de líneas, barras, histogramas, dispersión. Muy flexible y personalizable.
seaborn	No paper formal	Seaborn Docs	Visualización estadística avanzada y estética: heatmaps, boxplots, pairplots, countplots.
plotly	No paper formal (fundación Plotly)	Plotly Docs	Visualización interactiva en dashboards y web (gráficos dinámicos en 2D y 3D).
bokeh	Bryan et al. (2014)	Bokeh Paper	Visualizaciones interactivas y en tiempo real para la web.
altair	No paper formal	Altair Docs	Visualizaciones declarativas y rápidas basadas en la gramática de gráficos.
scipy	Virtanen et al. (2020)	Nature Paper	Funciones matemáticas avanzadas: álgebra lineal, optimización, estadísticas, integrales.
statsmodels	Seabold & Perktold (2010)	Statsmodels Paper	Modelos estadísticos clásicos: regresiones, ANOVA, tests de hipótesis, series temporales.
pingouin	Vallat (2018)	Pingouin Paper	Estadística avanzada simplificada: correlaciones, ANOVA, regresiones, tests.
scikit-learn	Pedregosa et al. (2011)	JMLR Paper	Machine Learning clásico: clasificación, regresión, clustering, reducción de dimensionalidad.
xgboost	Chen & Guestrin (2016)	XGBoost Paper	Algoritmo de gradient boosting muy eficiente y ganador de muchas competiciones ML.
lightGBM	Ke et al. (2017)	LightGBM Paper	Gradient boosting rápido y escalable en datasets grandes, de Microsoft.
catboost	Dorogush et al. (2018)	CatBoost Paper	Gradient boosting de Yandex optimizado para variables categóricas.
tensorflow	Abadi et al. (2016)	Tensorflow Paper	Framework de Deep Learning y ML a gran escala, muy usado en producción.
keras	Chollet (2015)	Keras Paper	API de alto nivel para construir redes neuronales de forma rápida y sencilla (sobre TensorFlow).
pytorch	Paszke et al. (2019)	PyTorch Paper	Framework flexible para Deep Learning. Muy usado en investigación y academia.
prophet	Taylor & Letham (2018)	Prophet Paper	Predicción de series temporales (forecasting) de forma sencilla.
tsfresh	Christ et al. (2018)	tsfresh Paper	Extracción automática de características relevantes en series temporales.
dask	Rocklin (2015)	Dask Paper	Computación paralela y distribuida en Python. Permite trabajar con datasets más grandes que la memoria.
pyspark	Zaharia et al. (2016)	Spark Paper	Procesamiento distribuido de grandes volúmenes de datos con Apache Spark.
vaex	Breddels & Veljanoski (2018)	Vaex Paper	Análisis rápido de datos masivos (<i>out-of-core</i> , sin cargar todo en memoria).

Mapa Visual de Selección

- **Carga de Datos:** pandas / dask / vaex / requests / beautifulsoup4 / selenium
- **Limpieza:** pandas / polars / datatable
- **EDA:** pandas-profiling / sweetviz / autoviz
- **Visualización:** matplotlib / seaborn / plotly
- **Modelado:** statsmodels / scikit-learn / xgboost / keras / pytorch
- **Series Temporales:** statsmodels / prophet / tsfresh
- **Big Data:** dask / pyspark / vaex

Snippets Comparativos de Código

pandas:

```
import pandas as pd
df = pd.read_csv('archivo.csv')
```

pandas

```
import pandas as pd
df = pd.read_csv('archivo.csv')
```

dask:

```
import dask.dataframe as dd
df = dd.read_csv('archivo.csv')
```

dask

```
import dask.dataframe as dd
df = dd.read_csv('archivo.csv')
```

vaex:

```
import vaex
df = vaex.open('archivo.csv')
```

vaex

```
import vaex
df = vaex.open('archivo.csv')
```

Web scraping:

```
import requests
from bs4 import BeautifulSoup
response = requests.get('https://url.gov')
soup = BeautifulSoup(response.text, 'html.parser')
```

Web scraping básico

```
import requests
from bs4 import BeautifulSoup
response = requests.get('https://url.gov')
soup = BeautifulSoup(response.text, 'html.parser')
```

Prophet:

```
from prophet import Prophet
```

```
model = Prophet()
```

```
model.fit(df)
```

prophet (forecast)

```
from prophet import Prophet
model = Prophet()
model.fit(df)
```

scikit-learn:

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)
```

scikit-learn (ML básico)

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

Glosario

- **Dataframe:** estructura tabular.
- **Array:** vector o matriz numérica.
- **EDA:** análisis exploratorio de datos.
- **Supervisado:** modelo con variable objetivo.
- **Boosting:** ensamble de modelos.
- **Out-of-core:** procesamiento sin cargar en RAM.
- **Cluster:** conjunto de máquinas distribuidas.

Actividad Práctica: 'Elige la Mejor Herramienta para el Análisis'

Objetivo: Desarrollar criterio en la selección de librerías según el problema de análisis.

Instrucciones:

- Elegir las librerías para cada fase: carga, limpieza, análisis, visualización y modelado.
- Justificar la elección.

Casos:

- 1) Encuesta pequeña (CSV con 800 encuestas de satisfacción de usuarios de un sistema público).
- 2) Big Data (logs de 150 millones de registros presupuestarios).
- 3) Forecast de ventas (recaudación fiscal diaria 5 años).
- 4) Clasificación con ML (contribuyentes: predecir cumplimiento).
- 5) Web Scraping (precios de contrataciones públicas).

Mini Quiz de Repaso

- 1) **Dataset 500MB:** ¿pandas o dask?
- 2) **Web con JavaScript:** ¿beautifulsoup o selenium?
- 3) **ARIMA:** ¿statsmodels o scikit-learn?
- 4) **Predicción de series sencilla:** ¿prophet o statsmodels?
- 5) **Boosting rápido:** ¿xgboost o tensorflow?